SPECYFIKACJA FUNKCJONALNA

Projekt: Wizualizacja grafu planarnego (część w języku C)

1. Wprowadzenie

Celem projektu jest stworzenie aplikacji konsolowej w języku C, której zadaniem jest wyznaczanie współrzędnych wierzchołków grafu planarnego w sposób umożliwiający jego estetyczną wizualizację.

W pierwszym etapie projektu aplikacja realizuje podstawowe funkcjonalności:
- wczytanie grafu z pliku tekstowego (dane testowe),
- przetworzenie danych wejściowych,
- wygenerowanie przykładowych współrzędnych wierzchołków,
- zapis wyników do pliku wyjściowego.

Na tym etapie nie są jeszcze zaimplementowane docelowe algorytmy rozmieszczania grafów — stosowane są dane testowe.

2. Zakres funkcjonalny systemu

Aplikacja umożliwia:
- odczyt grafu zapisanego w pliku tekstowym,
- reprezentację grafu w pamięci programu,
- generowanie przykładowych współrzędnych wierzchołków,
- zapis współrzędnych do pliku tekstowego lub binarnego,
- sterowanie programem za pomocą argumentów linii poleceń.

3. Sposób uruchomienia programu

Program uruchamiany jest z poziomu linii poleceń.

Ogólna składnia:
graph_layout -i <plik_wejściowy> [opcje]

4. Parametry wejściowe

- -i : plik wejściowy (wymagany)
- -o : plik wyjściowy (opcjonalny)
- -b : zapis binarny (opcjonalny)

5. Format danych wejściowych

<nazwa_krawędzi> <wierzchołek_A> <wierzchołek_B> <waga>

Przykład:
AB 1 2 1
BC 2 3 1
CD 3 4 1
DB 4 2 1.407

6. Format danych wyjściowych

Format tekstowy:
<wierzchołek> <x> <y>

Format binarny:
- liczba wierzchołków
- tablica struktur (id, x, y)

7. Struktura funkcjonalna systemu

Dane grafu przechowywane są w strukturze (graph.h) zawierającej listę wierzchołków oraz listę krawędzi, co umożliwia łatwe przetwarzanie danych w kolejnych etapach projektu.

```
#ifndef GRAPH_H
// Wierzchołek
typedef struct {
    int id;        // identyfikator wierzchołka
    double x;      // współrzędna X
    double y;      // współrzędna Y
} Vertex;


// Krawędź
typedef struct {
    char name[32]; // nazwa krawędzi
    int u;         // początek
    int v;         // koniec
    double weight; // waga
} Edge;


// Graf
typedef struct {
    Vertex* vertices; // tablica wierzchołków
    Edge* edges;      // tablica krawędzi
    int n_vertices;   // liczba wierzchołków
    int n_edges;      // liczba krawędzi
} Graph;

#endif
```

System składa się z dwóch modułów – wejścia i wyjścia.

Moduł parsowania wczytuje dane z pliku i tworzy strukturę grafu. Składa się on z plików pars.h i pars.c. W pliku pars.h znajduje się struktura nowego elementu oraz deklaracje funkcji przypisania nowego elementu i czytania pliku.

```
#ifndef PARS_H
#define PARS_H

typedef struct Element {
    edge dane;
    struct Element *next;
} Element;


Element* dodaj_element(Element *glowa, const char *name, int u, int v, double weight);
Element* czytaj_plik(int argc, char *argv[]);

#endif
```

Funkcja dodaj_element za pomocą malloca alokuje pamięć dla nowego elementu w dynamicznej tablicy, pola struktury wypełnia przekazanymi wartościami, a następnie ustawia go na początek listy.

```
Element* dodaj_element(Element *glowa, const char *name, int u, int v, double weight) {
    Element *nowy = malloc(sizeof(Element));

    if(nowy == NULL)
        return glowa;

    strncpy(nowy->dane.name, name, 31);
    nowy->dane.name[31] = '\0';
    nowy->dane.u = u;
    nowy->dane.v = v;
    nowy->dane.weight = weight;

    nowy->next=glowa;

    return nowy;

}
```

Funkcja czytaj_plik przyjmuje plik z danymi grafu. Jeżeli plik nie zostanie podany w linii poleceń program nie zadziała. Funkcja czyta plik i dopóki otrzymuje odpowiednie parametry zapisuje dane do dynamicznej tablicy, używając poprzednio zaimplementowanej funkcji. Po zakończeniu odczytu plik jest zamykany, a funkcja zwraca wskaźnik na pierwszy element stworzonej listy.

```
Element * czytaj_plik(int argc, char *argv[]){

    if(argc < 2)
        return NULL;

    FILE *f = fopen(argv[1], "r");

    if(f == NULL)
        return NULL;

    Element *glowa = NULL;

    int u,v;
    char name[32];
    double weight;

    while(fscanf(f,"%s %d %d %lf", name, &u, &v, &weight ) == 4) {
        glowa = dodaj_element(glowa, name, u,v,weight);
    }

    fclose(f);
    return glowa;
}
```

Moduł wyjścia odpowiada za zapisu danych do pliku tekstowego lub binarnego. Jest niezależny od sposobu generowania współrzędnych, dzięki czemu w przyszłości możliwe będzie łatwe podłączenie różnych algorytmów bez zmiany jego implementacji.

Plik output.h to interfejs modułu wyjścia, w którym deklarujemy funkcje i umożliwia nam korzystanie ze struktur:

```
#ifndef OUTPUT_H
#define OUTPUT_H

#include "graph.h"

// Generuje przykładowe (fake) współrzędne wierzchołków
void generate_fake_layout(Graph* g);

// Zapis do pliku tekstowego
int save_output_text(const char* filename, Graph* g);

// Zapis do pliku binarnego
int save_output_binary(const char* filename, Graph* g);

#endif
```

W pliku output.c inicjalizujemy wszystkie funkcje podane w output.h.

Wczytanie grafu z pliku tekstowego (na tym etapie dane mogą być generowane testowo). Ta część będzie zastąpiona algorytmami w dalszych etapach projektu, wykorzystuje ona funkcje trygonometryczne sinus i cosinus, co pozwala na równomierne rozmieszczenie punktów w przestrzeni dwuwymiarowej.

```
void generate_fake_layout(Graph* g) {
    if (g == NULL || g->n_vertices == 0)
        return;

    double radius = 10.0;

    for (int i = 0; i < g->n_vertices; i++) {
        double angle = 2.0 * M_PI * i / g->n_vertices;

        g->vertices[i].x = radius * cos(angle);
        g->vertices[i].y = radius * sin(angle);
    }
}
```

Zapis danych realizowany jest w dwóch formatach: tekstowym oraz binarnym. Format tekstowy umożliwia łatwą analizę wyników przez użytkownika, natomiast format binarny zapewnia większą wydajność i mniejszy rozmiar pliku.

```c
// ----------------------------------
int save_output_text(const char* filename, Graph* g) {
    if (g == NULL)
        return 0;

    FILE* f = fopen(filename, "w");
    if (f == NULL) {
        printf("Error: cannot open file %s\n", filename);
        return 0;
    }

    for (int i = 0; i < g->n_vertices; i++) {
        fprintf(f, "%d %.3f %.3f\n",
                g->vertices[i].id,
                g->vertices[i].x,
                g->vertices[i].y);
    }

    fclose(f);
    return 1;
}
```

```c
//
int save_output_binary(const char* filename, Graph* g) {
    if (g == NULL)
        return 0;

    FILE* f = fopen(filename, "wb");
    if (f == NULL) {
        printf("Error: cannot open file %s\n", filename);
        return 0;
    }

    // zapis liczby wierzchołków
    fwrite(&g->n_vertices, sizeof(int), 1, f);

    // zapis tablicy Vertex
    fwrite(g->vertices, sizeof(Vertex), g->n_vertices, f);

    fclose(f);
    return 1;
}
```

8. Przebieg działania programu

1. Uruchomienie programu
2. Wczytanie grafu
3. Utworzenie struktury danych
4. Generowanie współrzędnych
5. Zapis do pliku
6. Zakończenie działania